FINAL REPORT: TI616 - Sustainable Digital Project

Team Members: Anthony Varela, Louis Demoy, Tristan Pichard
GitHub Repository: https://github.com/MegaiSa/Sustainable-digital-project.git
Live Demo URL: https://sustainabledigital.vendredicorp.fr/

1st Part

Step 1:

User need

People want to test and deepen their knowledge in an engaging, gamified way — without wading through dense articles.

Type of website

Educational gamified platform — a quiz game with scoring, progression, and rewards.

Target audience

Students, educators, and elders who want to learn about new topics in a fun, interactive format.

Value to users

Learn sustainability facts through play, track personal progress, compete on a leaderboard, and discover actionable tips after each quiz.

Why realistic for 4–5 students

The core mechanics (quiz engine, user auth, scoring) are well-understood and achievable with standard web technologies. Roles can be split naturally: 1 backend, 1–2 frontend, 1 UI/design.

Step 2:

Core function

A registered user can take a quiz, receive a score, and see their results compared against other users on a leaderboard. As well as create their own quiz.

Essential

User registration & login Quiz player (MCQ format)

Score calculation & display Leaderboard (top scores)

Admin — add/edit questions and tips after each answer

Quiz categories

Nice to have

Badges & achievements

Daily challenge mode Multiplayer

Social sharing

Explanations with external links

Step 3:

Visitor (guest)

View homepage & leaderboard
Browse quiz categories
Cannot play (must register)
No create / edit / delete

Player (registered user)

Play quizzes
View own history & score
Cannot manage others questions
Cannot delete others' data

Admin

All player permissions
Create / edit questions
Manage categories

Delete questions & users

Step 4:

User

id (UUID)
username
email
hashed password
role (guest / player / admin)
created_at

Avoid: phone number, address, date of birth, profile photo (v1)

Question

id (UUID)
question_text
choices (array, 4 options)
correct_answer_index
category_id (FK)
difficulty (easy / medium / hard)

Quiz session

id
user_id (FK)
category_id (FK)
score
total questions
completed at

Avoid: storing per-answer timing data

Category

id
name
description
icon_name

Step 5

User management
As a visitor, I want to create an account with my email and a username, so that I can save my quiz scores and appear on the leaderboard.

Benefit: Persistent identity enables progression tracking and competition.

User management
As a registered player, I want to log in securely and view my score history, so that I can track my improvement over time.

Benefit: Motivation through visible progress encourages return visits.

Quiz content
As an admin, I want to create, edit, and delete quiz questions with multiple-choice answers.

Benefit: Content quality is maintained without developer intervention.

Quiz content
As a player, I want to choose a quiz category and answer 10 randomised questions, then see my score.

Benefit: Learning happens through immediate feedback linked to real-world actions.

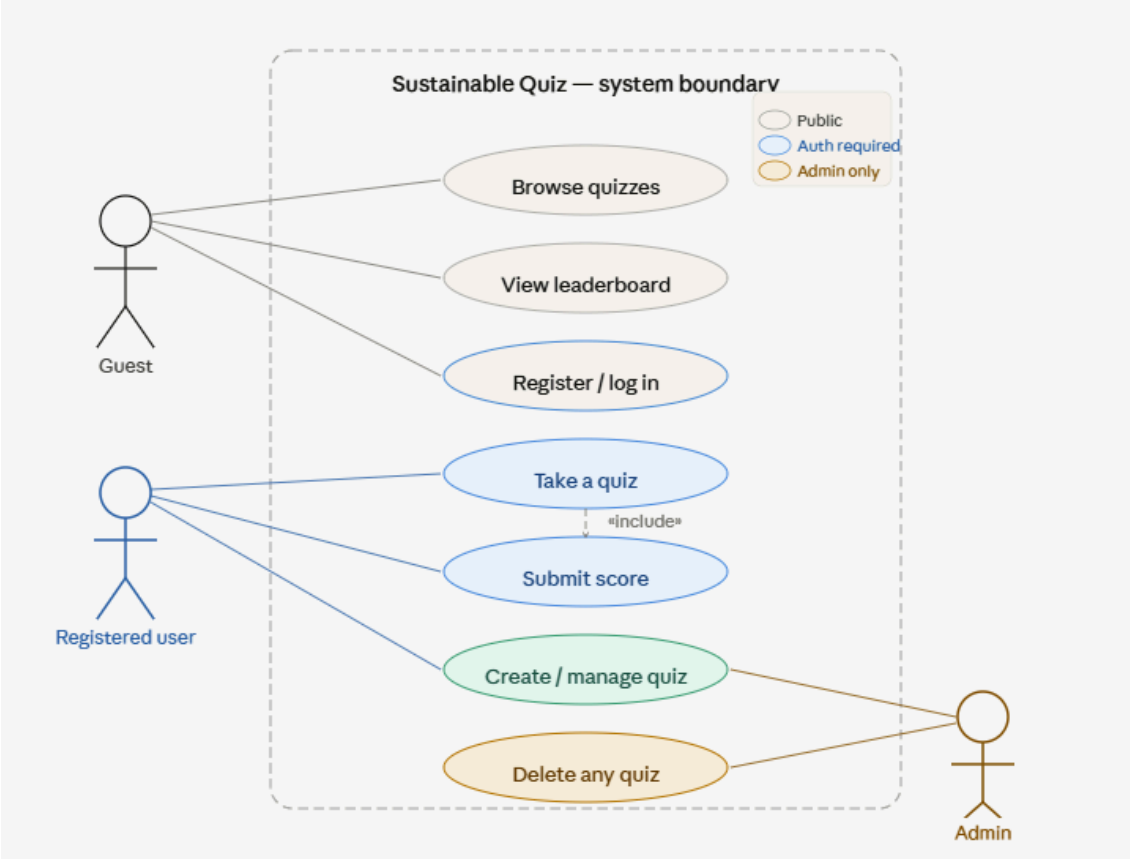
Step 6:

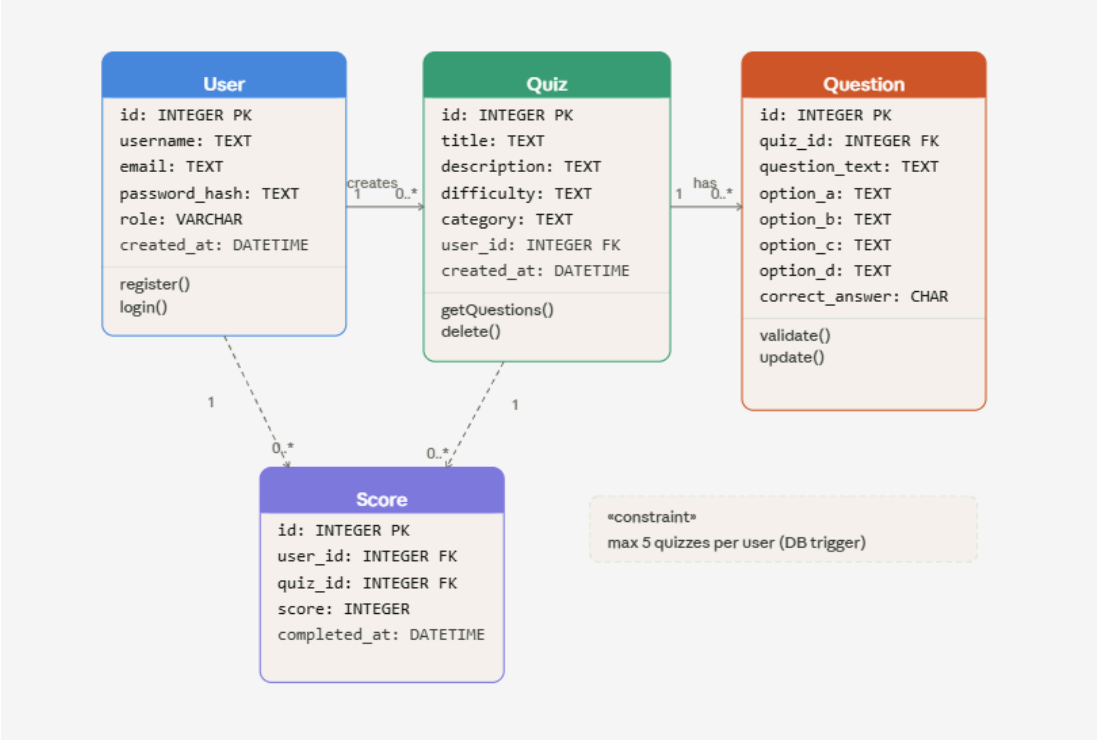
Priority	Element	Status
P1 (High)	As a user, I want to create an account with a secure password so that my data is protected.	Done
P1 (High)	As a user, I want to log in to my dashboard to access my personalized quiz features.	Done
P1 (High)	As a user, I want to log in to my dashboard to	Done

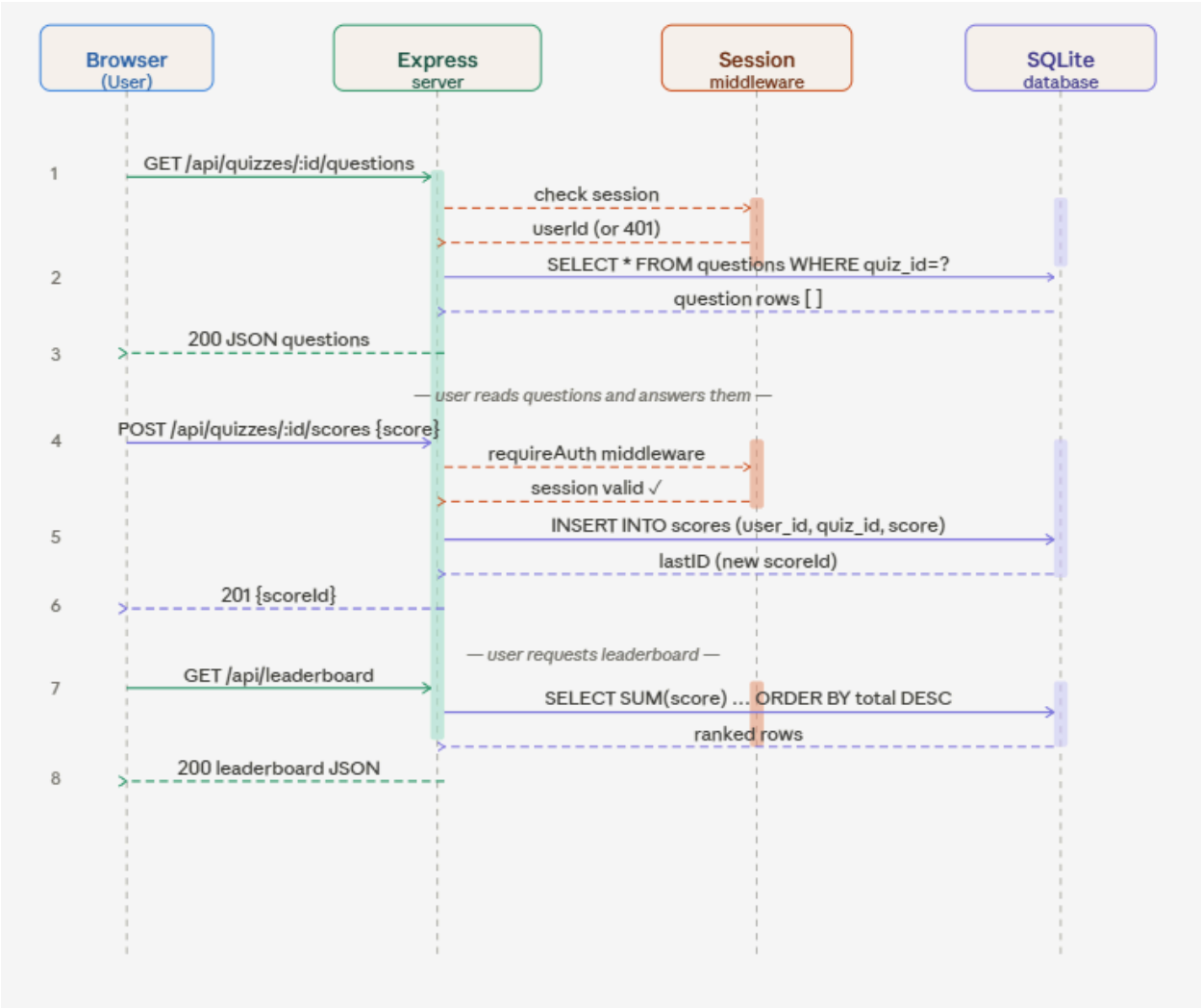
	access my personalized quiz features.	
P2 (Medium)	As a teacher, I want to create a new quiz with specific questions to share with the class.	Done
P2 (Medium)	As a developer, I want to limit each user to 5 quizzes maximum to prevent "Dark Data" and DB bloat.	Done
P2 (Medium)	As a user, I want to see a leaderboard of the top 10 scores to compare my performance.	Done
P2 (Medium)	As a developer, I want to refactor the database to use user_id globally to optimize SQL joins.	Done
P3 (Low)	As an admin, I want to delete old quizzes to ensure the database remains lightweight (Green IT).	Done
P3 (Low)	As a user, I want a minimalist UI without heavy images to reduce the page weight (EcoIndex).	Done
P3 (Low)	As a developer, I want to run Lighthouse audits to	To Do

	ensure the FCP is under 1.5 seconds.	

Step 7:




User
id: INTEGER PK
username: TEXT
email: TEXT
password_hash: TEXT
role: VARCHAR
created_at: DATETIME
register()
login()
creates
1
0..*
Quiz
id: INTEGER PK
title: TEXT
description: TEXT
difficulty: TEXT
category: TEXT
user_id: INTEGER FK
created_at: DATETIME
getQuestions()
delete()
1
has
0..*
Question
id: INTEGER PK
quiz_id: INTEGER FK
question_text: TEXT
option_a: TEXT
option_b: TEXT
option_c: TEXT
option_d: TEXT
correct_answer: CHAR
validate()
update()
1
0..*
0..*
1
Score
id: INTEGER PK
user_id: INTEGER FK
quiz_id: INTEGER FK
score: INTEGER
completed_at: DATETIME
«constraint»
max 5 quizzes per user (DB trigger)



2nd Part

---

1. Introduction

This project was carried out as part of the Sustainable Digital module. It consists of a quiz web application designed according to the principles of digital eco-design. The main objective was to provide a complete user experience while drastically minimizing the carbon footprint through a "Lean" technology stack and optimized server resource management.

2. Analysis & Design (Consistency with Part 1)

The architecture was designed to be lean from the design phase:

- **Use Case Diagram:** Focus on essential features (creation, participation, leaderboard) to avoid "software bloat".
- **Class Diagram:** Simplified relational structure to limit the complexity of SQL queries.

3. Final Technology Choices & Justification

Each technology was chosen for its energy efficiency:

Component	Technology	Green IT Justification
Back-end	Node.js / Express	Ultra-lightweight asynchronous execution, ideal for limiting CPU usage.
Database	SQLite3	"Serverless" database. No background process, energy consumption only during access.
Front-end	HTML5 / CSS3 / Vanilla JS	No heavy frameworks (React/Angular). Minimal loading time and JS parsing reduced to the strict minimum.
Security	Bcrypt	Optimal balance between robust security and CPU cycles required for hashing.

4. Detailed Implementation

4.1 Database Optimization

4.1 Front-end Development & Optimization

The front-end was developed with a focus on Visual Sobriety and Resource Efficiency, ensuring high performance on all types of devices.

- Native Stack: We used Vanilla HTML5, CSS3, and JavaScript. By avoiding heavy frameworks (like React or Vue), we minimized the browser's CPU overhead and significantly reduced the total page weight.
- Asset Sobriety:
    - Zero External Dependencies: The project does not rely on external libraries, Google Fonts, or heavy CDNs. This reduces the number of HTTP requests to less than 15 per page, meeting the Green IT targets.
    - Text-Based Interface: We prioritized a clean, text-based CSS design over heavy images to minimize data transfer.
- Optimized Data Flow:
    - On-Demand Fetching (Conditional Rendering): Data is only requested from the API when needed (e.g., loading a specific quiz). This prevents the transfer of unnecessary data, reducing network energy consumption.
      +1
    - Efficient Execution: JavaScript logic is triggered only after the DOM is ready, ensuring a fast First Contentful Paint (FCP) and a smooth user experience even on low-end hardware.
- Accessibility: Use of semantic HTML tags (<nav>, <main>, <footer>) to ensure the site is lightweight, easy to parse by browsers, and accessible to all users.

4.2 Business Logic & Sessions

The backend is built with Node.js and the Express.js framework, following a RESTful architecture. The codebase is split into two main files: server.js, which handles all routing and middleware configuration, and db.js, which encapsulates every database interaction. This separation of concerns keeps the code clean and makes each layer independently maintainable. The database itself is SQLite, initialised at startup by executing init.sql, which creates the four core tables — users, quizzes, questions, and scores — if they do not already exist.

Express is configured with three middleware layers: express.json() and express.urlencoded() to parse incoming request bodies, and express.static() to serve the frontend files directly from the server. Session management is handled by express-session, which stores the authenticated user's ID, username, and role server-side. Cookies are set as httpOnly to prevent client-side JavaScript access, with a one-hour expiry. A custom requireAuth middleware function is applied to all write-operation routes; it checks for an active session and returns a 401 Unauthorized response if none is found, ensuring unauthenticated users cannot create, modify, or delete any data.

The API exposes thirteen endpoints across four functional groups. The authentication group covers registration, login, logout, and session check (/api/me). The quiz group allows clients to list all quizzes, create a new one, or delete an existing one. The question group supports fetching, adding, editing, and deleting questions for a given quiz. Finally, the scoring group handles score submission after a quiz is completed and exposes a global leaderboard aggregated by total score across all quizzes.

Security is enforced at multiple levels throughout the API:

- Password hashing: bcrypt is used with a cost factor of 12, meaning passwords are salted and hashed before storage and never saved in plaintext.
- Ownership checks: before any mutation (edit or delete) on a quiz or question, the server queries the database to confirm the requesting user is the resource owner. Administrators bypass this check via a role stored in the session.
- Parameterised queries: all SQL statements in db.js use ? placeholders, preventing SQL injection by ensuring user input is always treated as data, never as executable code.
- Input validation: every route verifies that required fields are present and correctly formatted before reaching the database, returning descriptive 400 errors otherwise.

From an ecological perspective, SQLite was deliberately chosen over a client-server database engine such as PostgreSQL or MySQL. For a project of this scale, running a dedicated database server would consume unnecessary CPU cycles and memory. SQLite operates as a single file accessed directly by the Node process, eliminating that overhead entirely. Additionally, performance indexes defined in init.sql on the key foreign key columns (user_id, quiz_id, category) ensure that queries remain efficient as data grows, avoiding costly full-table scans that would increase processing time and energy consumption.

5. Carbon Analysis & Environmental Impact

The application was audited on its final deployed version.

Measurement Tool	Metric	Result	
EcoIndex	Score / Grade	96/100	
Website Carbon	CO2 / View	A+	

Major Optimization: Implementation of a **SQL Trigger** limiting to a maximum of 5 quizzes per

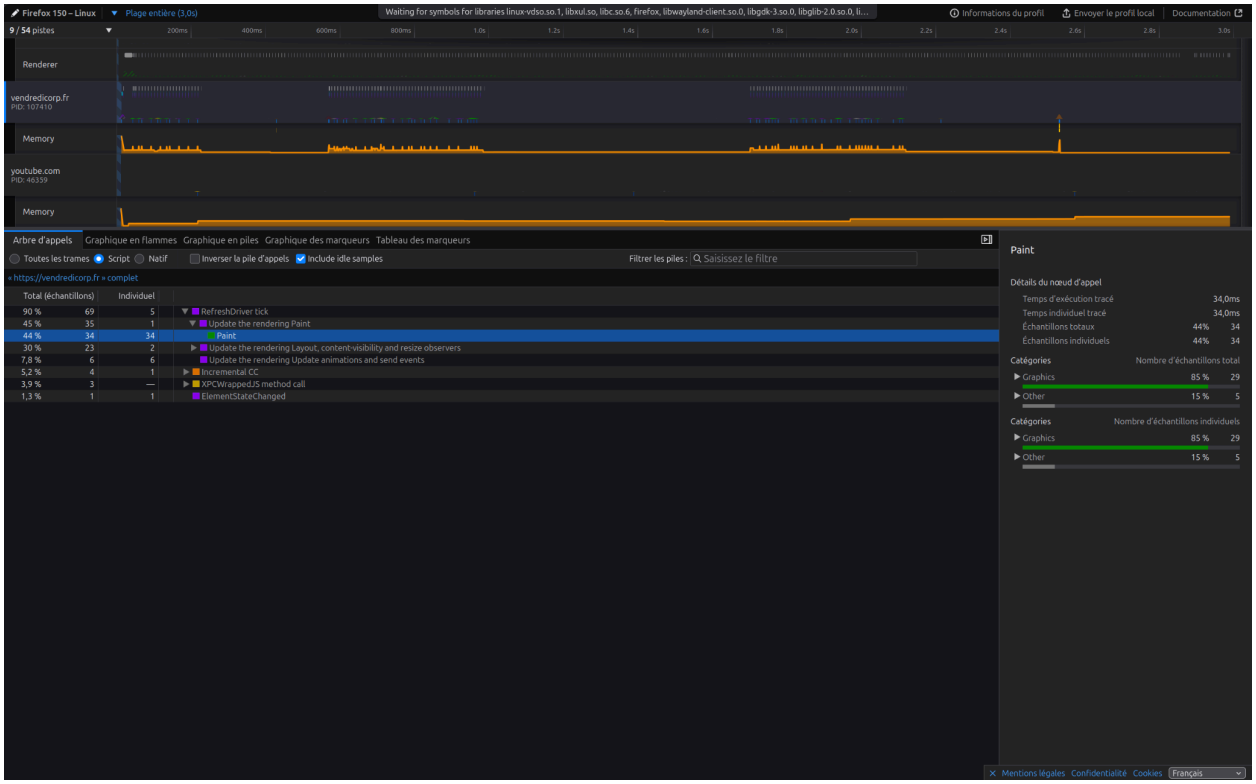
user. This direct eco-design measure intentionally limits the growth of the database to preserve server resources.

6. Testing & Quality

6.1 Functional Test Suite

ID	Description	Expected Result	Status
T01	Registration/Login	Bcrypt hashing and session OK	PASS
T02	Quota Limit	6th quiz rejected by the Trigger	PASS
T03	Owner Access	Modification forbidden to third parties	PASS

6.2 Lighthouse Reports



7. Project Management

We used the Kanban method via GitHub Projects. Commits are distributed evenly, reflecting collaboration on code refactoring (fixing the user_id bug) and integrating Green IT features.

8. Conclusion

The project allowed us to demonstrate that it is possible to reconcile an interactive user experience with digital sobriety. By prioritizing simple technologies (SQLite, Vanilla JS) and intelligent SQL constraints, we achieved our minimal impact goals while ensuring a robust and secure application.